

老子软考，扫码下载



老子
软考

老子软考
软考用老子

软件设计师-2018年上半年-案例题

第1题：阅读下列说明，回答问题1至问题4，将解答填入答题纸的对应栏内。

【说明】

某医疗护理机构为老年人或有护理需求者提供专业护理，现欲开发一基于Web的医疗管理系统，以改善医疗护理效率。该系统的主要功能如下：

- (1) 通用信息查询。客户提交通用信息查询请求，查询通用信息表，返回查询结果。
- (2) 医生聘用。医生提出应聘 / 辞职申请，交由主管进行聘用/解聘审批，更新医生表，并给医生反馈聘用/解聘结果；删除解聘医生的出诊安排。
- (3) 预约处理。医生安排出诊时间，存入医生出诊时间表；根据客户提交的预约查询请求，查询在职医生及其出诊时间等预约所需数据并返回；创建预约，提交预约请求，在预约表中新增预约记录，更新所约医生出诊时间并给医生发送预约通知；给客户反馈预约结果。
- (4) 药品管理。医生提交处方，根据药品名称从药品数据中查询相关药品库存信息，开出药品，更新对应药品的库存以及预约表中的治疗信息；给医生发送“药品已开出”反馈。
- (5) 报表创建。根据主管提交的报表查询请求（报表类型和时间段），从预约数据、通用信息、药品库存数据、医生以及医生出诊时间中进行查询，生成报表返回给主管。

现采用结构化方法对医疗管理系统进行分析与设计，获得如图1-1所示的上下文数据流图和图1-2所示的0层数据流图。

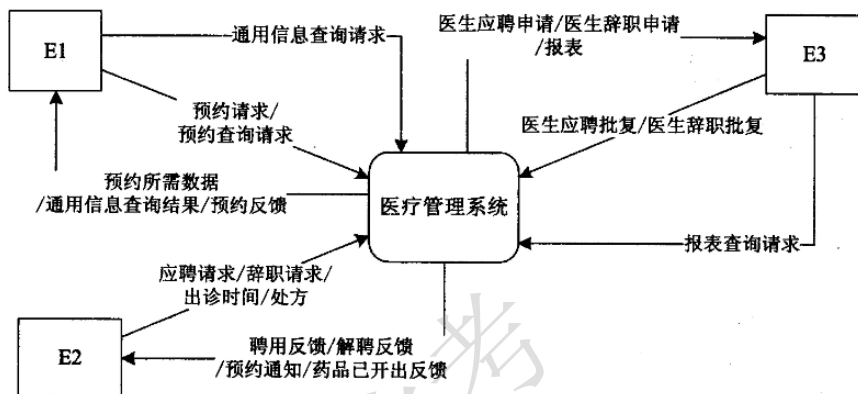


图 1-1 上下文数据流图

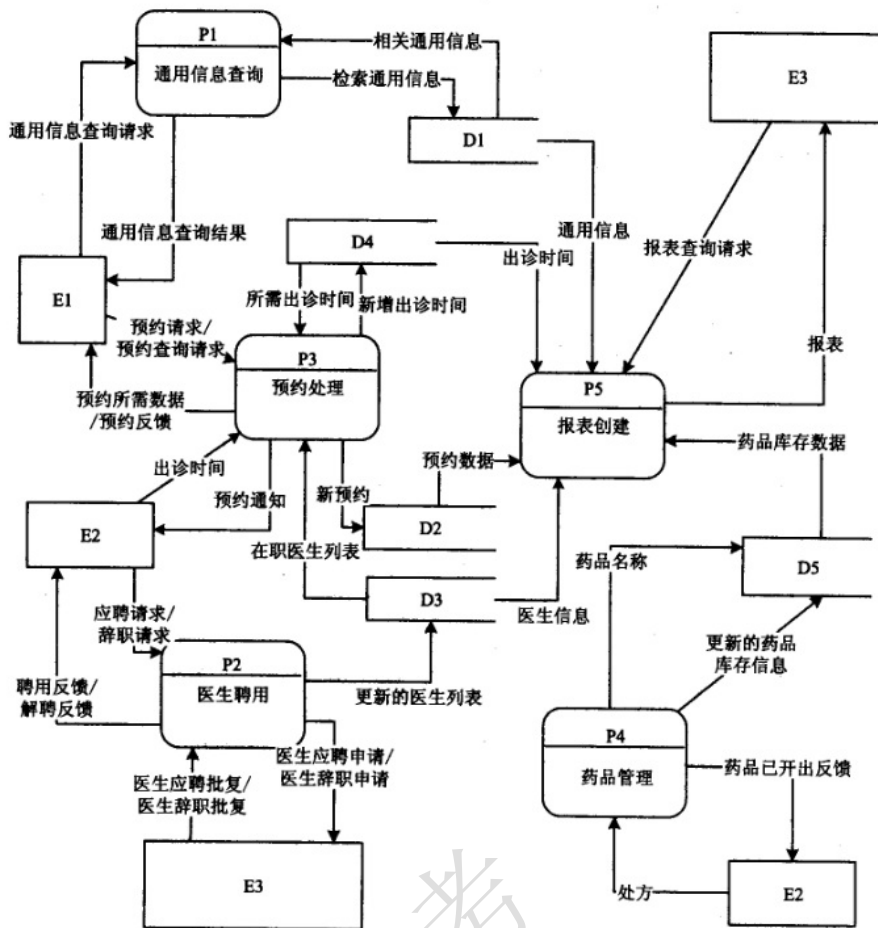


图 1-20 层数据流图

题：1.1 （3分）

使用说明中的词语，给出图1-1中的实体E1～E3的名称。

题：1.2 （5分）

使用说明中的词语，给出图1-2中的数据存储D1～D5的名称。

题：1.3 （4分）

使用说明和图中术语，补充图1-2中缺失的数据流及其起点和终点。

题：1.4 （3分）

使用说明中的词语，说明“预约处理”可以分解为哪些子加工，并说明建模图1-1和图1-2是如何保持数据流图平衡。

第2题：阅读下列说明，回答问题1至问题3，将解答填入答题纸的对应栏内。

【说明】

某海外代购公司为扩展公司业务，需要开发一个信息化管理系统。请根据公司现有业务及需求完成该系统的数据库设计。

【需求描述】

- (1) 记录公司员工信息。员工信息包括工号、身份证号、姓名、性别和一个手机号，工号唯一标识每位员工，员工分为代购员和配送员。
- (2) 记录采购的商品信息。商品信息包括商品名称、所在超市名称、采购价格、销售价格和商品介绍，系统内部用商品条码唯一标识每种商品。一种商品只在一家超市代购。
- (3) 记录顾客信息。顾客信息包括顾客真实姓名、身份证号（清关缴税用）、一个手机号和一个收货地址，系统自动生成唯一的顾客编号。
- (4) 记录托运公司信息。托运公司信息包括托运公司名称、电话和地址，系统自动生成唯一的托运公司编号。
- (5) 顾客登录系统之后，可以下单购买商品。订单支付成功后，系统记录唯一的支付凭证编号，顾客需要在订单里指定运送方式：空运或海运。
- (6) 代购员根据顾客的订单在超市采购对应商品，一份订单所含的多个商品可能由多名代购员从不同超市采购。
- (7) 采购完的商品交由配送员根据顾客订单组合装箱，然后交给托运公司运送。托运公司按顾客订单核对商品名称和数量，然后按顾客的地址进行运送。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图2-1所示。

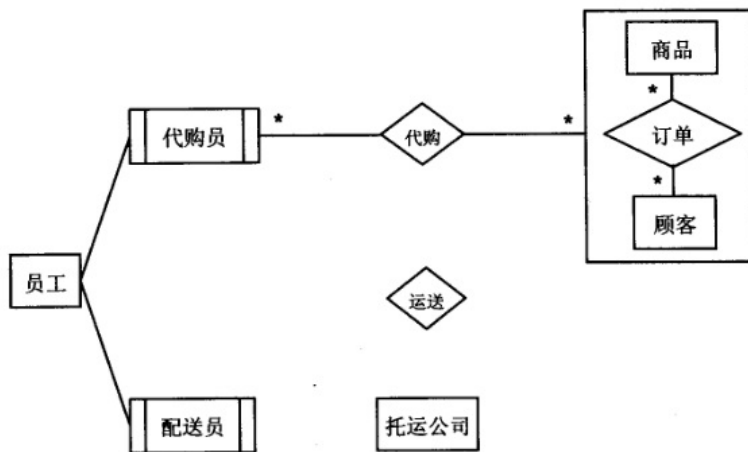


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

员工（工号，身份证号，姓名，性别，手机号）

商品（条码，商品名称，所在超市名称，采购价格，销售价格，商品介绍）

顾客（编号，姓名，身份证号，手机号，收货地址）

托运公司（托运公司编号，托运公司名称，电话，地址）

订单（订单ID，（a），商品数量，运送方式，支付凭证编号）

代购（代购ID，代购员工号，（b））

运送（运送ID，配送员工号，托运公司编号，订单ID，发运时间）题：2.1 （3分）

根据问题描述，补充图2-1的实体联系图。

题：2.2 （6分）

补充逻辑结构设计结果中的（a）、（b）两处空缺。

题：2.3 （6分）

为方便顾客，允许顾客在系统中保存多组收货地址。请根据此需求，增加“顾客地址”弱实体，对图2-1进行补充，并修改“运送”关系模式。

第3题：阅读下列说明，回答问题1至问题3，将解答填入答题纸的对应栏内。

【说明】

某ETC（Electronic Toll Collection，不停车收费）系统在高速公路沿线的特定位置上设置一个横跨道路上空的龙门架（Toll gantry），龙门架下包括6条车道（Traffic lanes），每条车道上安装有雷达传感器（Radar sensor）、无线传输器（Radio transceiver）和数码相机（Digital Camera）等用于不停车收费的设备，以完成正常行驶速度下的收费工作。该系统的基本工作过程如下：

（1）每辆汽车上安装有车载器，驾驶员（Driver）将一张具有唯一识别码的磁卡插入车载器中。磁卡中还包含有驾驶员账户的当前信用记录。

（2）当汽车通过某条车道时，不停车收费设备识别车载器内的特有编码，判断车型，将收集到的相关信息发送到该路段所属的区域系统（Regional center）中，计算通行费用，创建收费交易（Transaction），从驾驶员的专用账户中扣除通行费用。如果驾驶员账户透支，则记录透支账户交易信息。区域系统再将交易后的账户信息发送到维护驾驶员账户信息的中心系统（Central system）

（3）车载器中的磁卡可以使用邮局的付款机进行充值。充值信息会传送至中心系统，以更新驾驶员账户的余额。

（4）当没有安装车载器或者车载器发生故障的车辆通过车道时，车道上的数码相机将对车辆进行拍照，并将车辆照片及拍摄时间发送到区域系统，记录失败的交易信息；并将该交易信息发送到中心系统。

（5）区域系统会获取不停车收费设备所记录的交通事件（Traffic events）；交通广播电台（Traffic advice center）根据这些交通事件进行路况分析并播报路况。

现采用面向对象方法对上述系统进行分析与设计，得到如表3-1所示的用例列表以及如图3-1所示的用例图和图3-2所示的分析类图。

表 3-1 用例列表

用例名称	说明
Create transaction	记录收费交易
Charge card	磁卡充值
Underpaid transaction	记录透支账户交易信息
Record Illegal use	记录失败交易信息
Record traffic event	记录交通事件

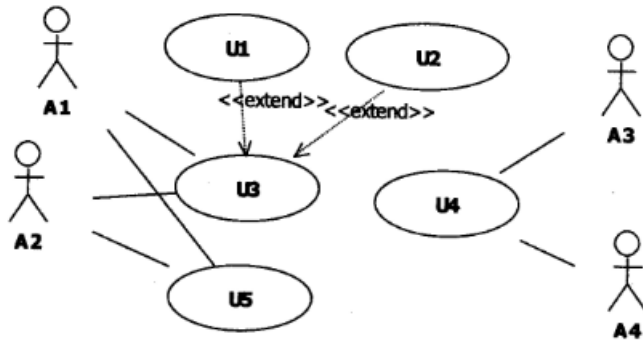


图 3-1 用例图

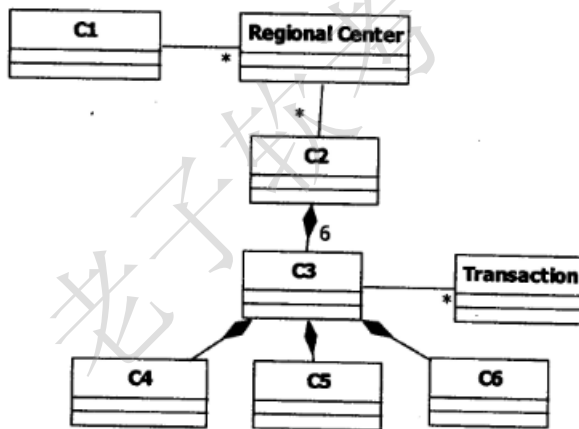


图 3-2 分析类图

题：3.1 （4分）

根据说明中的描述，给出图3-1中A1~A4所对应的参与者名称。

题：3.2 （5分）

根据说明中的描述及表3-1，给出图3-1中U1~U5所对应的用例名称。

题：3.3 （6分）

根据说明中的描述，给出图3-2中C1~C6所对应的类名。

第4题：阅读下列说明和C代码，回答问题1和问题2，将解答填入答题纸的对应栏内。

【说明】

某公司购买长钢条，将其切割后进行出售。切割钢条的成本可以忽略不计，钢条的长度为整英寸。已知价格表 p ，其中 p_i ($i=1, 2, \dots, m$) 表示长度为 i 英寸的钢条的价格。现要求解使销售收益最大的切割方案。

求解此切割方案的算法基本思想如下：

假设长钢条的长度为 n 英寸，最佳切割方案的最左边切割段长度为 i 英寸，则继续求解剩余长度为 $n-i$ 英寸钢条的最佳切割方案。考虑所有可能的 i ，得到的最大收益 r_n 对应的切割方案即为最佳切割方案。 r_n 的递归定义如下：

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

对此递归式，给出自顶向下和自底向上两种实现方式。

【C代码】

/* 常量和变量说明

n: 长钢条的长度

```
p[]: 价格数组
*/
#define LEN 100

int Top_Down_Cut_Rod(int p[],int n){ /*自顶向下*/
    int r=0;
    int i;
    if(n == 0){
        return 0;
    }
    for(i=1; (1) ;i++){
        int tmp = p[i]+Top_Down_Cut_Rod(p,n-i);
        r=(r>=tmp)?r: tmp;
    }
    return r;
}

int Bottom_Up_Cut_Rod(int p[],int n){ /*自底向上*/
    int r[LEN]={0};
    int temp=0;
    int i,j;
    for(j=1;j<=n;j++){
        temp=0;
        for(i=1; (2) ;i++){
            temp= (3) ;
        }
        (4) ;
    }
    return r[n];
}
```

题：4.1 （8分）
根据说明，填充C代码中的空（1）～（4）。
题：4.2 （7分）
根据说明和C代码，算法采用的设计策略为（5）。
求解 r_n 时，自顶向下方方法的时间复杂度为（6）；自底向上方法的时间复杂度为（7）（用O表示）。

第5题：阅读下列说明和C++代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】
生成器（Builder）模式的意图是将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。图5-1所示为其类图。

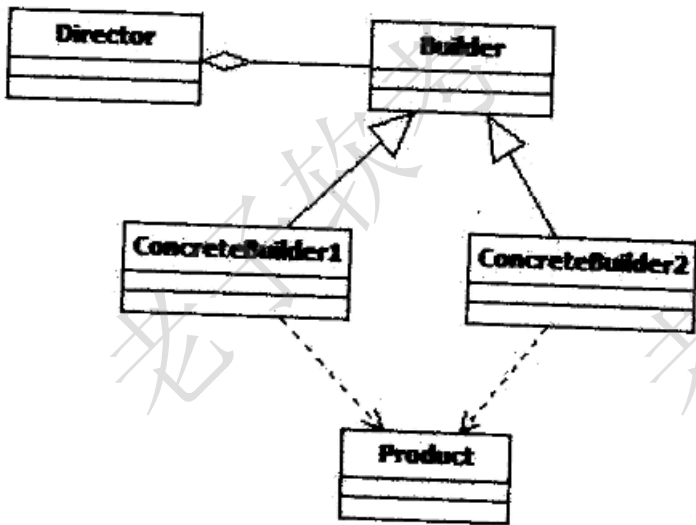


图 5-1 生成器模式类图

【C++代码】

```
#include <iostream>
#include <string>
using namespace std;

class Product {
private:
    string partA, partB;
public:
    Product() { }
    void setPartA(const string& s) { PartA = s;}
    void setPartB(const string& s) { PartB = s;}
    // 其余代码省略
};

class Builder {
public:
    (1) ;
    virtual void buildPartB()=0;
    (2) ;
};

class ConcreteBuilder1 : public Builder {
private:
    Product* product;
public:
    ConcreteBuilder1() {product = new Product(); }
    void buildPartA() { (3) ("Component A"); }
    void buildPartB() { (4) ("Component B"); }
    Product* getResult() { return product; }
    // 其余代码省略
};

class ConcreteBuilder2 : public Builder {

    /* 代码省略 */

};

class Director {
private:
    Builder* builder;
public:
    Director(Builder* pBuilder) { builder= pBuilder;}
    void construct() {
        (5) ;
        // 其余代码省略
    }
    // 其余代码省略
};

int main() {
    Director* director1 = new Director(new ConcreteBuilder1());
    director1->construct();
    delete director1;
    return 0;
}题： 5.1
```

第6题：阅读下列说明和Java代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

生成器（Builder）模式的意图是将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。图6-1所示为其类图。

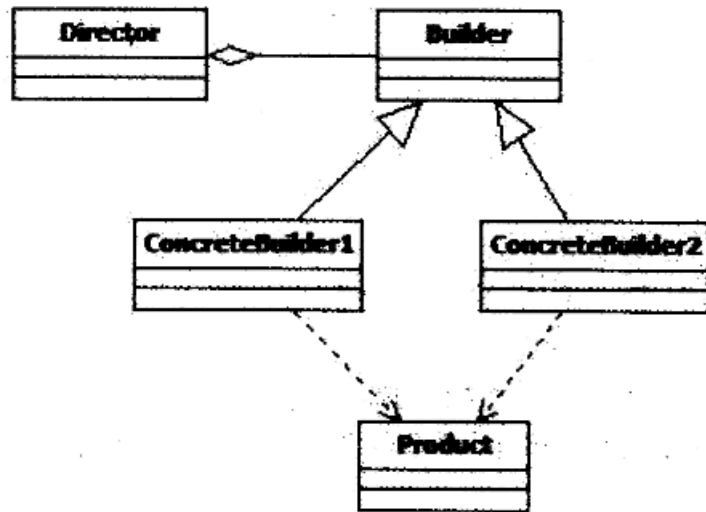


图 6-1 生成器模式类图

【Java代码】

```
import java.util.*;
```

```
class Product {
    private String partA;
    private String partB;
    public Product() {}
    public void setPartA(String s) { partA = s; }
    public void setPartB(String s) { partB = s; }
}

interface Builder {
    public    (1)    ;
    public void buildPartB();
    public    (2)    ;
}

class ConcreteBuilder1 implements Builder {
    private Product product;
    public ConcreteBuilder1() { product = new Product(); }
    public void buildPartA() {    (3)    ("Component A"); }
    public void buildPartB() {    (4)    ("Component B"); }
    public Product getResult() { return product; }
}

class ConcreteBuilder2 implements Builder {

    // 代码省略
}

class Director {
    private Builder builder;
    public Director(Builder builder) {this.builder = builder; }
    public void construct() {
        (5)    ;
        // 代码省略
    }
}

class Test {
    public static void main(String[] args) {
        Director director1 = new Director(new ConcreteBuilder1());
        director1.construct();
    }
}

}题： 6.1
```

